

ASSOCIATION PATTERN MINING

DV2627 – Advanced Machine Learning



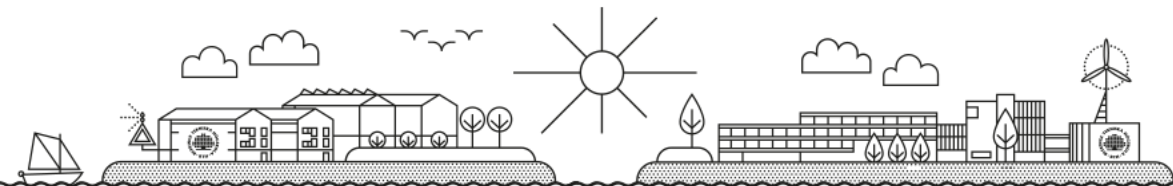
Shahrooz Abghari
Computer Science Department

January 28, 2025



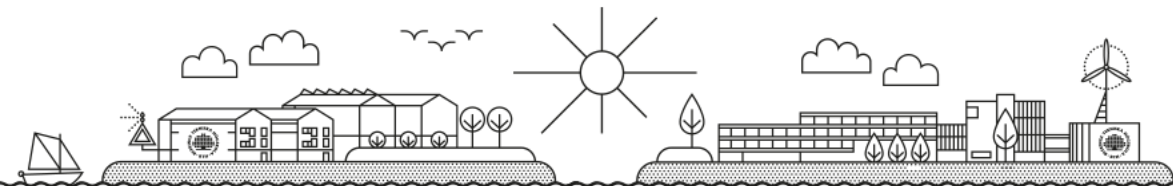
OUTLINE

- **Pattern Discovery: What is it? Why is it important?**
- The Frequent Pattern Mining Model
- Association Rule Generation Framework
- Frequent Itemset Mining Algorithms
- Useful Meta-algorithms



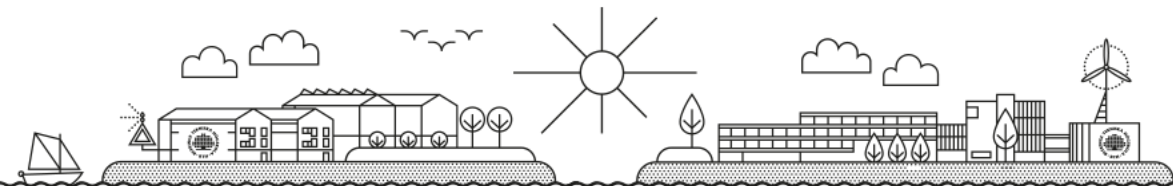
WHAT IS PATTERN DISCOVERY?

- **Pattern**: a set of items, subsequences, substructures, etc., that frequently occurs in a data set.
- Patterns usually represent **inherent** and **important properties** of data
- **Pattern discovery**: uncovering patterns from massive datasets
- First proposed by Agrawal et al. [1] in the context of **frequent itemsets** and **association rule mining**
- Motivational examples:
 - What products were often purchased together? (Target marketing)
 - What are the subsequent purchases after buying a PC? (Customer behavior analysis)
 - What word sequences likely form phrases? (Text analysis)



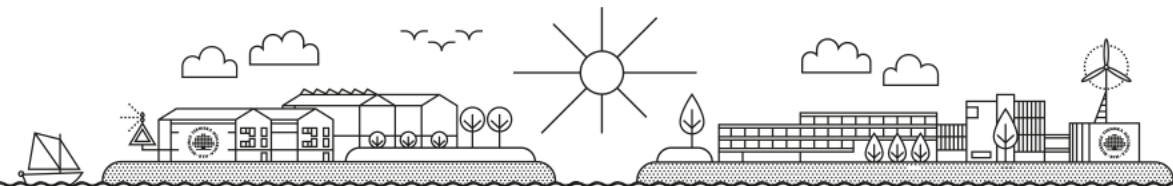
PATTERN DISCOVERY IMPORTANCE

- Finding **inherent regularities** in the data sets
- Foundation for many essential data mining tasks
 - Association, correlation, and causality analysis,
 - Mining sequential, structural (e.g., sub-graphs) patterns,
 - Pattern analysis is spatiotemporal data, multimedia data, and stream data.
 - Cluster analysis: pattern-based subspace clustering
- Broad applications:
 - Market basket analysis, cross-marketing, sale campaign analysis, web log analysis, biological sequence analysis



SOME TERMINOLOGIES

- **Transactions:** Sets of items bought by customers
- **The goal:** Determine associations between groups of items bought by customers
- Quantification of the level of **association** is based on frequencies of sets of items.
- Discovered sets of items are known as:
 - Large itemsets, frequent itemsets, or frequent patterns

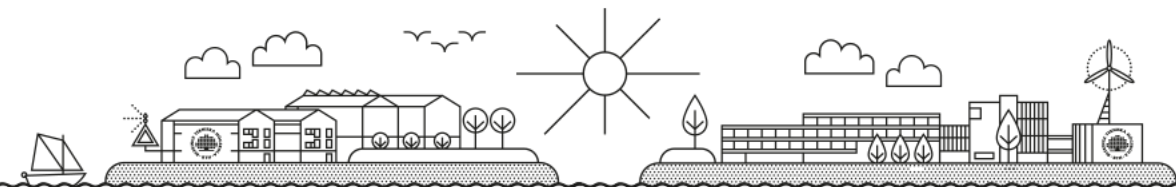


THE FREQUENT PATTERN MINING MODEL

- U is a set of d items
- T is a set of n transactions $T_1, \dots, T_n$
 - $T_i \subseteq U$
- Binary Representation of $T_1, \dots, T_n$
 - $U = \{\text{Beer, Coffee, Diaper, Eggs, Milk, Nuts,}\}$
 - $T_1 = \{101001\}$
- **itemset**: A set of one or more items
- **k-itemset** $X = \{x_1, \dots, x_k\}$
- **(absolute) support**, or **support count** of X : Frequency or occurrence of an itemset X
- **(relative) support**, s , is the fraction of transactions that contain X (i.e., the **probability** that a transaction contains X)
- An itemset X is **frequent** if X 's support is no less than a **minsup** threshold.

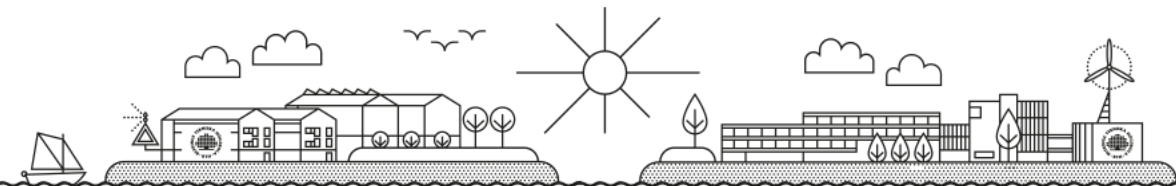
Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk

- Let minsup = 50%
- Freq. 1-itemsets:
 - **Beer**: 3 (60%); **Nuts**: 3 (60%)
 - **Diaper**: 4 (80%); **Eggs**: 3 (60%)
- Freq. 2-itemsets:
 - **{Beer, Diaper}**: 3 (60%)



CHALLENGE: THERE ARE TOO MANY FREQUENT PATTERNS!

- A long pattern contains a combinatorial number of sub-patterns
- How many frequent itemsets does the following TDB contain?
 - TDB: $T_1: \{a_1, \dots, a_{50}\}; T_2: \{a_1, \dots, a_{100}\}$
 - Assuming (absolute) minsup = 1
 - Let's have a try
 - **1-itemsets:** $\{a_1\}: 2, \{a_2\}: 2, \dots, \{a_{50}\}: 2, \{a_{51}\}: 1, \dots, \{a_{100}\}: 1,$
 - **2-itemsets:** $\{a_1, a_2\}: 2, \dots, \{a_{49}, a_{50}\}: 2, \{a_1, a_{51}\}: 1 \dots, \dots, \{a_{99}, a_{100}\}: 1,$
 - ...
 - **99-itemsets:** $\{a_1, a_2, \dots, a_{99}\}: 1, \dots, \{a_2, a_3, \dots, a_{100}\}: 1$
 - **100-itemsets:** $\{a_1, a_2, \dots, a_{100}\}: 1$
 - In total: $\binom{100}{1} + \binom{100}{2} + \dots + \binom{100}{100} = 2^{100} - 1$ sub-patterns



EXPRESSING PATTERNS IN COMPRESSED FORM: *CLOSED PATTERNS*

○ How to handle such a challenge?

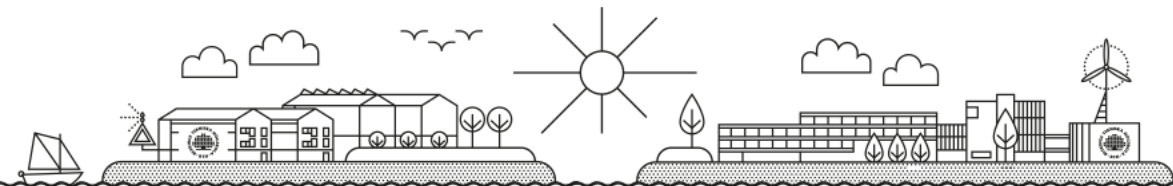
○ **Solution 1:**

Closed patterns: A pattern (itemset) X is **closed** if X is frequent, and there exists **no super-pattern $Y \supset X$, with the same support as X**

- Let TDB: $T_1: \{a_1, \dots, a_{50}\}$; $T_2: \{a_1, \dots, a_{100}\}$
- Suppose minsup = 1. How many closed patterns does TDB contain?
 - Two: $P_1: \{\{a_1, \dots, a_{50}\}: 2\}$; $P_2: \{\{a_1, \dots, a_{100}\}: 1\}$

○ **Closed pattern** is a **lossless compression** of frequent patterns

- Reduces the number of patterns but does not lose the support information!
- You will still be able to say: $\{\{a_2, \dots, a_{40}\}: 2\}$, $\{\{a_5, a_{51}\}: 1\}$



EXPRESSING PATTERNS IN COMPRESSED FORM: *MAX-PATTERNS*

○ Solution 2:

Max-patterns: A pattern X is a **max-pattern** if X is frequent and there exists **no frequent super-pattern, $Y \supset X$**

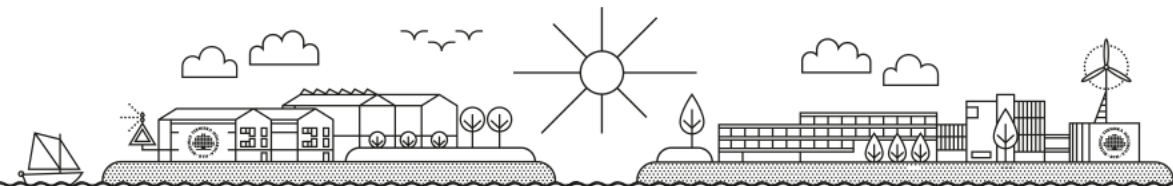
Difference from close-patterns?

- Do not care about the real support of the sub-patterns of a max-pattern
- Let TDB: $T_1: \{a_1, \dots, a_{50}\}$; $T_2: \{a_1, \dots, a_{100}\}$
- Suppose minsup = 1. How many max-patterns does TDB contain?
 - One: $P: \{a_1, \dots, a_{100}\}: 1$

○ **Max-pattern** is a **lossy compression**!

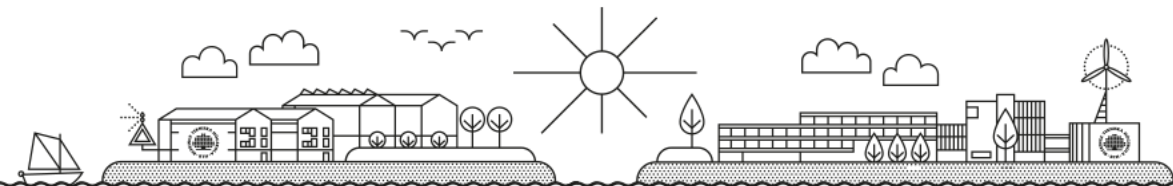
- We only know $\{a_1, \dots, a_{40}\}$ is frequent
- But we do not know the real support of $\{a_1, \dots, a_{40}\}$, ..., anymore!

○ Thus in many applications, mining close-patterns is more desirable than mining max-patterns



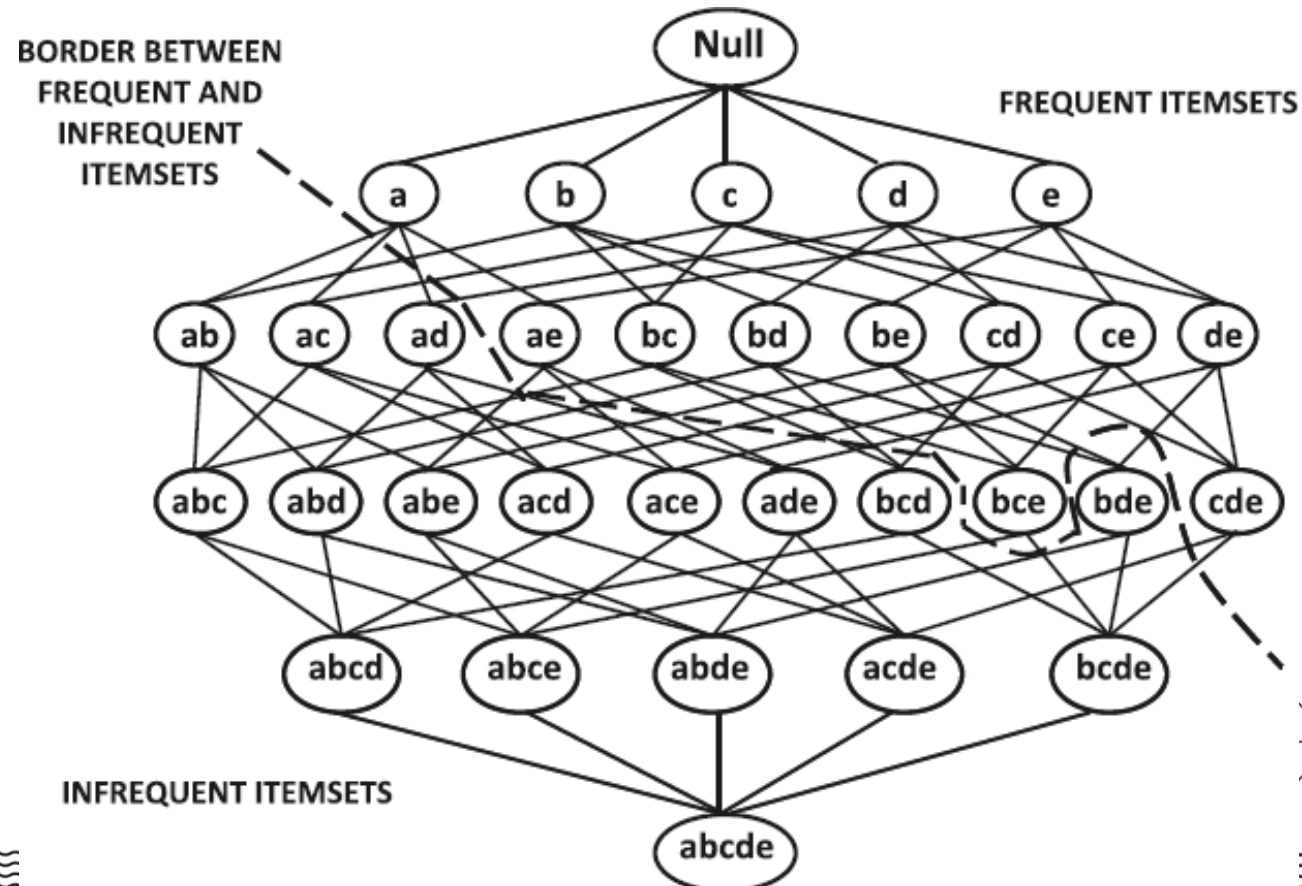
DOWNWARD CLOSURE PROPERTY

- Observation: From TDB: $T_1: \{a_1, \dots, a_{50}\}$; $T_2: \{a_1, \dots, a_{100}\}$
 - We get a frequent itemset (**minsup = 2**): $\{a_1, \dots, a_{50}\}$
 - Also, its subsets are all frequent: $\{a_1\}$, $\{a_2\}$, ..., $\{a_{50}\}$, $\{a_1, a_2\}$, ..., $\{a_1, \dots, a_{49}\}$, ...
 - There must be some hidden relationships among frequent patterns!
- The **downward closure (also called "Apriori")** property of frequent patterns
 - If **{Beer, Nuts, Diaper}** is frequent, so is **{Beer, Diaper}**
 - Every transaction containing **{Beer, Nuts, Diaper}** also contains **{Beer, Diaper}**
 - Apriori: Any subset of a frequent itemset must be frequent
- Efficient mining methodology
 - If **any subset of an itemset X** is infrequent, then there is no chance for X to be frequent—why do we even have to consider X!? **Good mechanism for pruning**



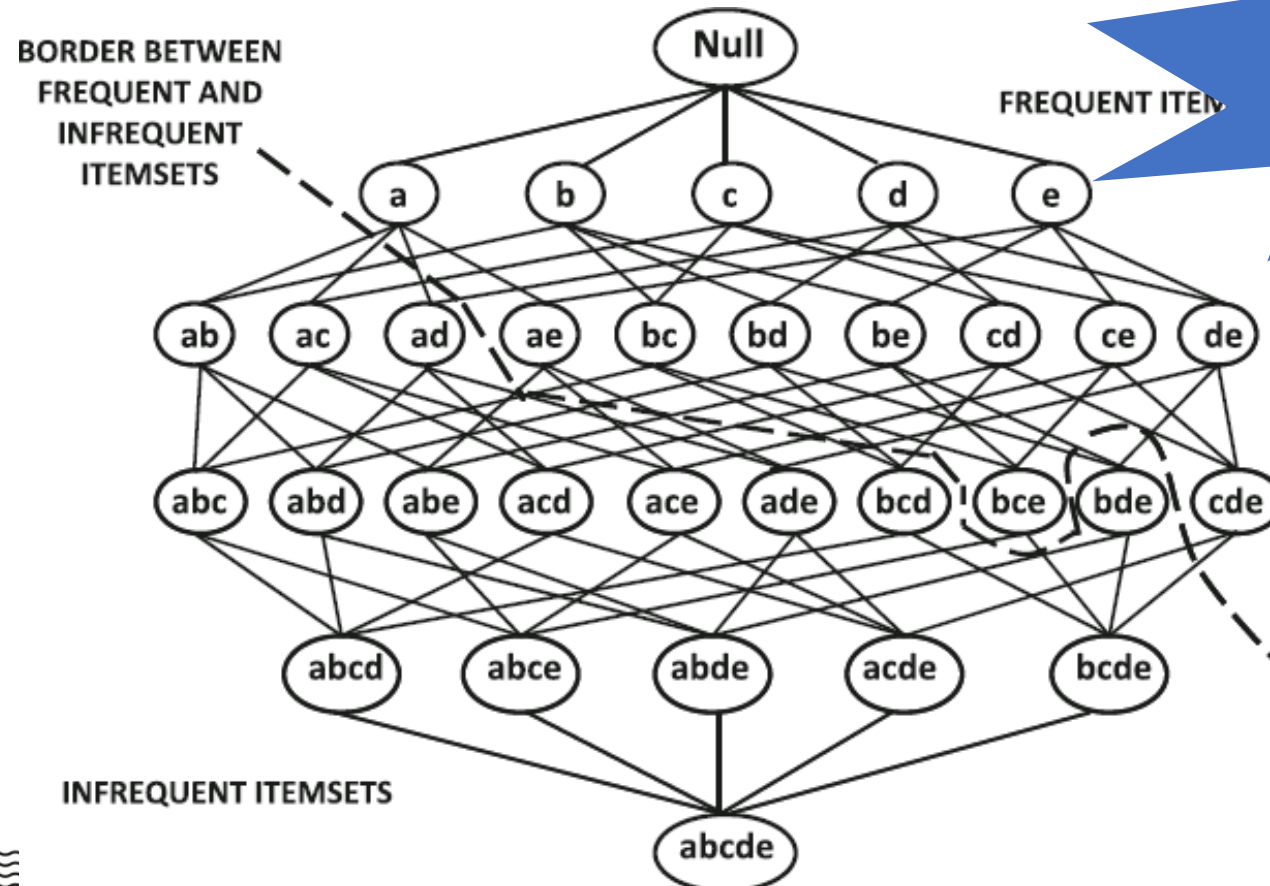
THE ITEMSET LATTICE

- Contain $2^{|U|}$ nodes and represents search space

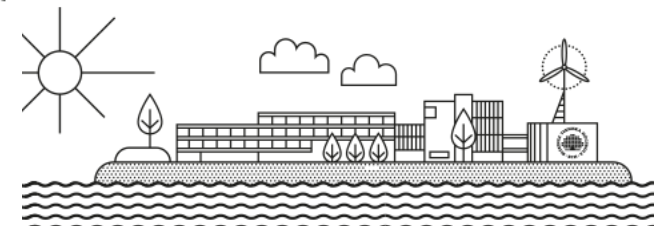


THE ITEMSET LATTICE

- Contain $2^{|U|}$ nodes and represents search space

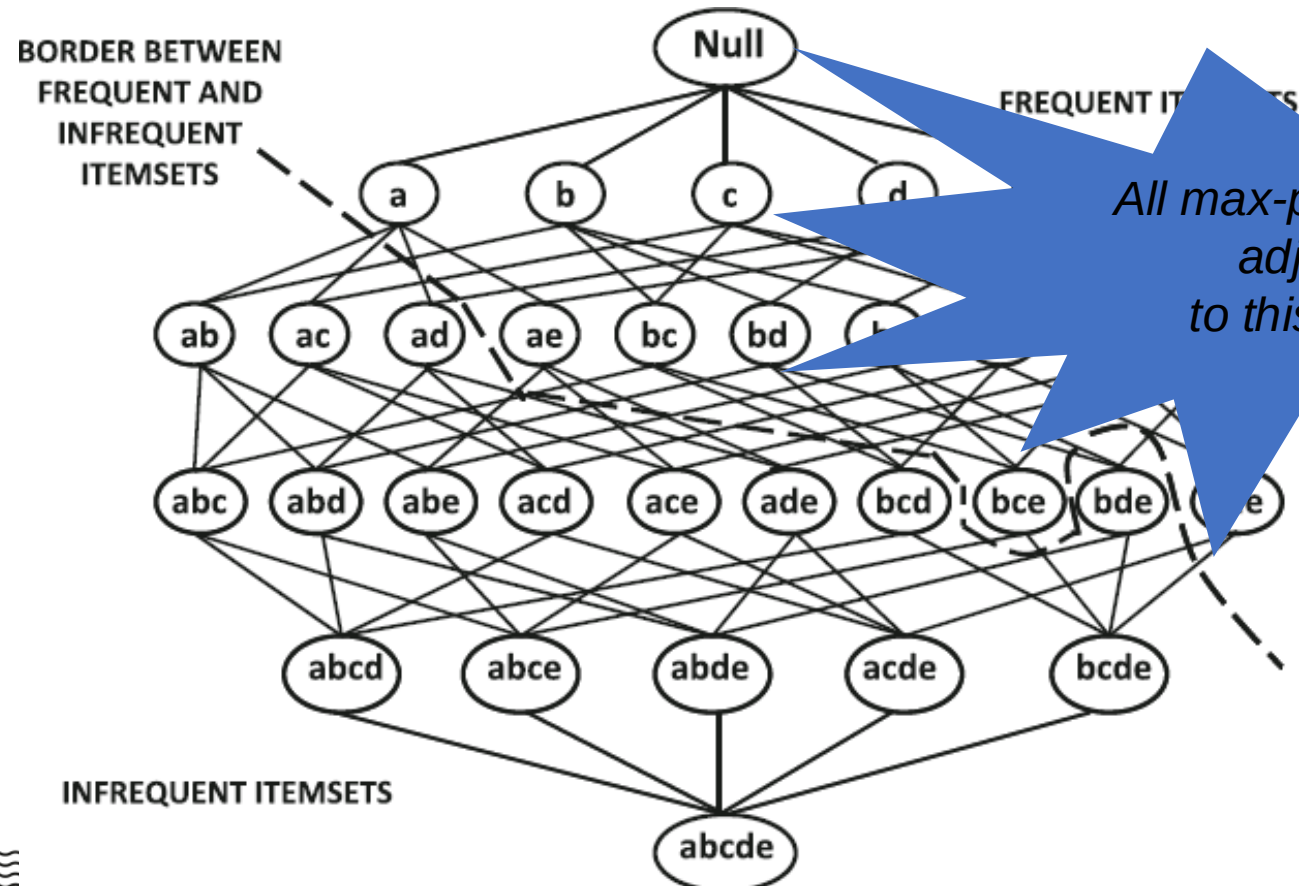


All itemsets above this border are frequent.

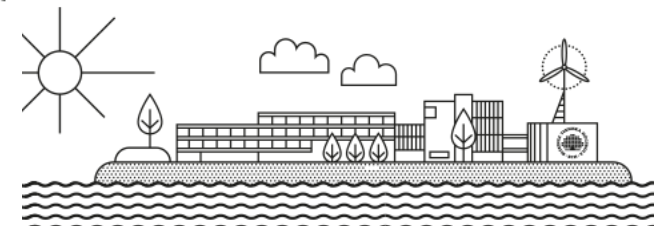


THE ITEMSET LATTICE

- Contain $2^{|U|}$ nodes and represents search space

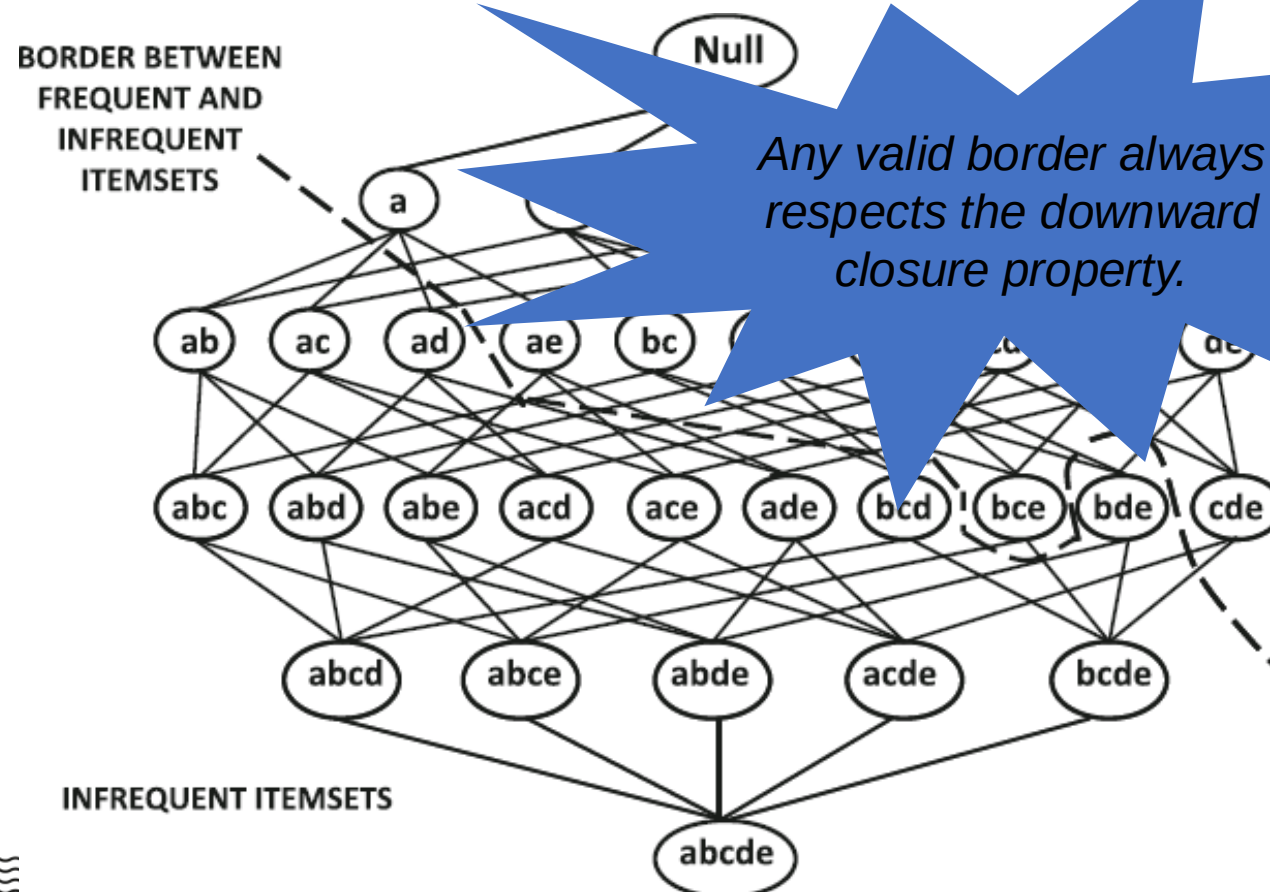


All max-patterns are adjacent to this border



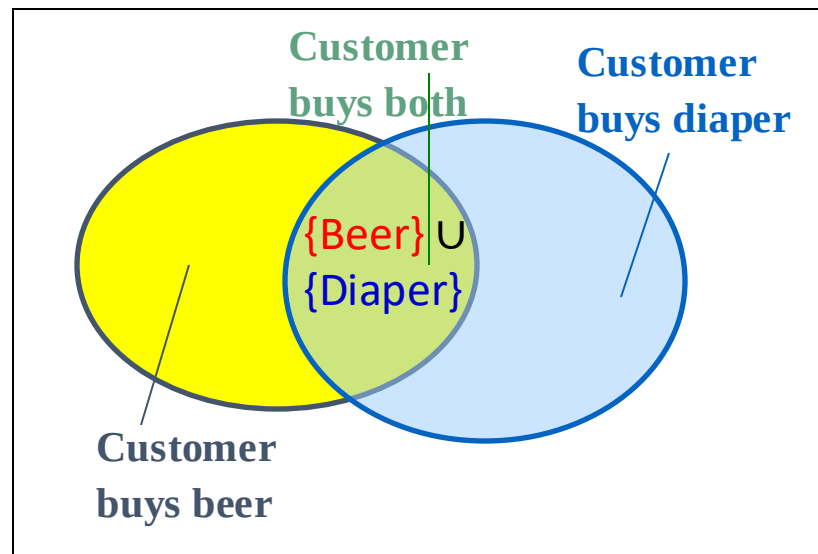
THE ITEMSET LATTICE

- Contain $2^{|U|}$ nodes and represents search space



FROM FREQUENT ITEMSETS TO ASSOCIATION RULES

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk

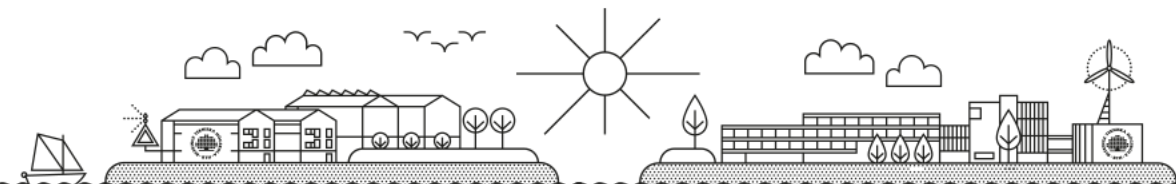


- Find all the rules $X \rightarrow Y(s,c)$ with minimum support and confidence
 - support, s , probability that a transaction contains $X \cup Y$
 - confidence, c , the conditional probability that a transaction having X also contains Y

$$c = \text{Sup}(Y|X) = \frac{\text{Sup}(X \cup Y)}{\text{Sup}(X)}$$

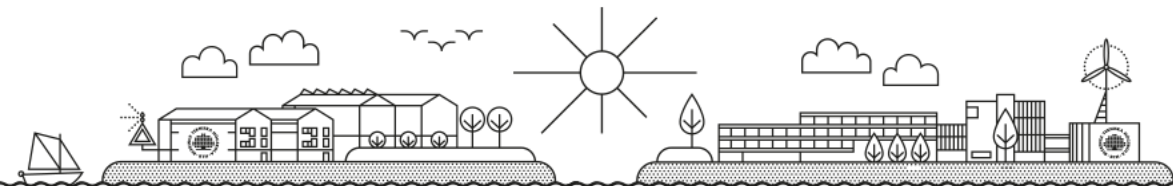
- Association rule mining: Find all of the rules, $X \rightarrow Y$, with minimum support and confidence
- Let minsup = 50%,
 - Freq. 1-itemsets: Beer:3 (60%), Nuts:3, Diaper:4 (80%), Eggs:3 (60%),
 - Freq. 2-itemsets: {Beer, Diaper}:3 (60%)

- Association rules: minconf = 50%
 - Beer $\rightarrow$ Diaper (60%, 100%) = $(3/5)/(3/5) = 1$
 - Diaper $\rightarrow$ Beer (60%, 75%) = $(3/5)/(4/5) = 0.75$



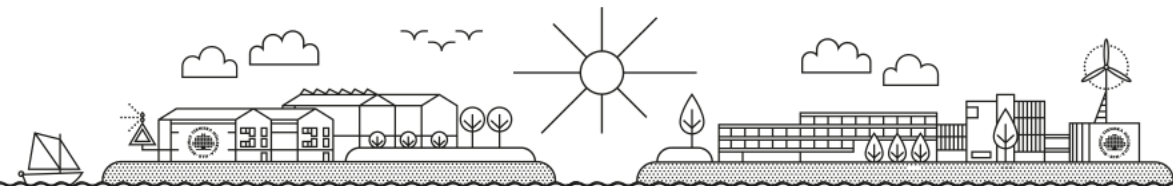
THE OVERALL FRAMEWORK FOR ASSOCIATION RULE MINING

1. In the *1st phase*, all the frequent itemsets are generated at the **minimum support of *minsup***.
 - The most difficult step
2. In the *2nd phase*, the association rules are generated from the frequent itemsets at the **minimum confidence level of *minconf***.
 - Relatively straightforward



IMPLEMENTATION OF 2ND PHASE

- A Straightforward Implantation
 1. Given a frequent itemset I
 2. Generate all possible partitions X and $Y = I - X$
 3. Examine the confidence of each $X \Rightarrow Y$



IMPLEMENTATION OF 2ND PHASE

○ Reduce the Search Space

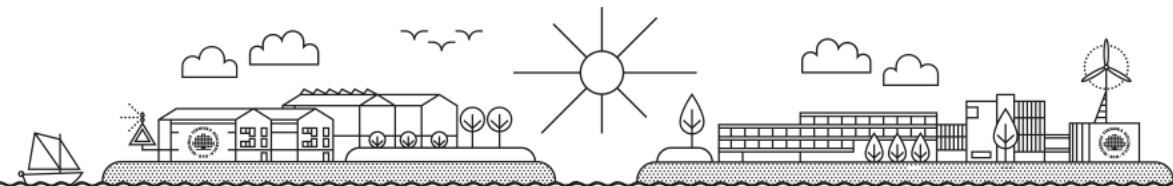
Property 4.3.1 (Confidence Monotonicity) *Let X_1 , X_2 , and I be itemsets such that $X_1 \subset X_2 \subset I$. Then the confidence of $X_2 \Rightarrow I - X_2$ is at least that of $X_1 \Rightarrow I - X_1$.*

$$\text{conf}(X_2 \Rightarrow I - X_2) \geq \text{conf}(X_1 \Rightarrow I - X_1) \quad (4.3)$$

{**Bread**} $\rightarrow$ {Butter, **Milk**}

{**Bread**, Butter} $\rightarrow$ {**Milk**}

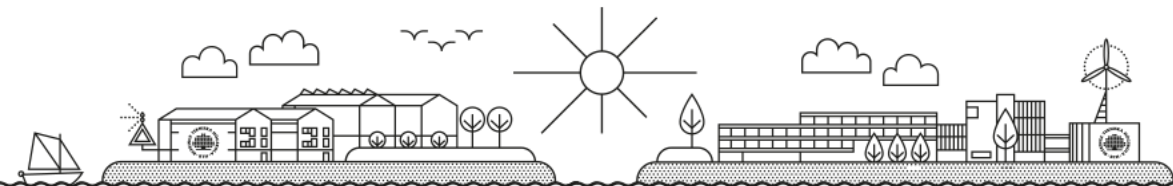
2nd rule is redundant concerning the first; it will have the same support but a confidence that is no less than the first.



FREQUENT ITEMSET MINING ALGORITHMS



- **Brute Force Algorithms**
- Apriori Algorithm
- FP-Growth Algorithm



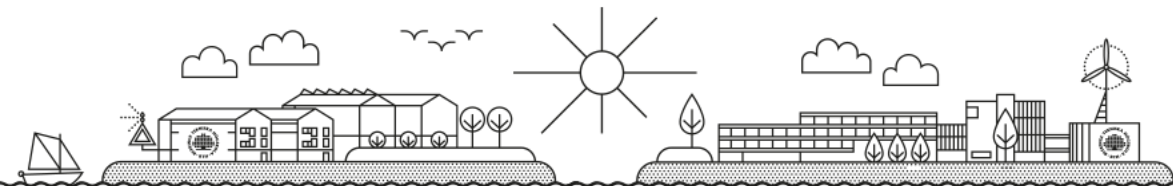
BRUTE FORCE ALGORITHMS (1)

○ The Naïve Approach

- Generate all these candidate itemsets
 - For a universe of items U , there are a total of $2^{|U|} - 1$ distinct subsets
 - When $U = 1000$, $2^{1000} \geq 10^{300}$
- Count their support against the transaction database

○ Observation

- no $(k + 1)$ -patterns are frequent if no k - patterns are frequent
(**downward closure property**).



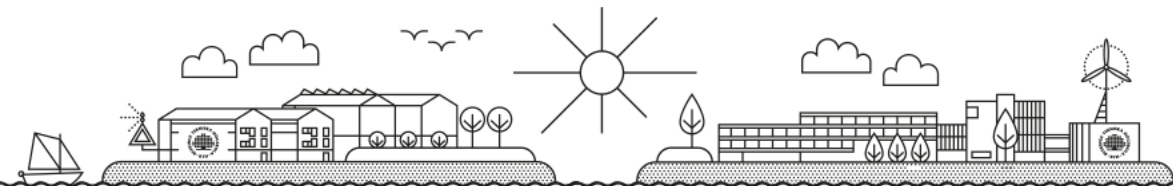
BRUTE FORCE ALGORITHMS (2)

- An improved approach
 1. Generate all candidate k -itemsets
 2. Count their support against the transaction database
 3. If no frequent itemsets are found, then stop; Otherwise, $k++$ and continue;
- A significant improvement
 - Let l be the final value of k

$$\sum_{i=1}^l \binom{|u|}{i} \ll 2^{|u|}$$

- $|u| = 1000$ and $l = 10$, it is order of 10^{23}

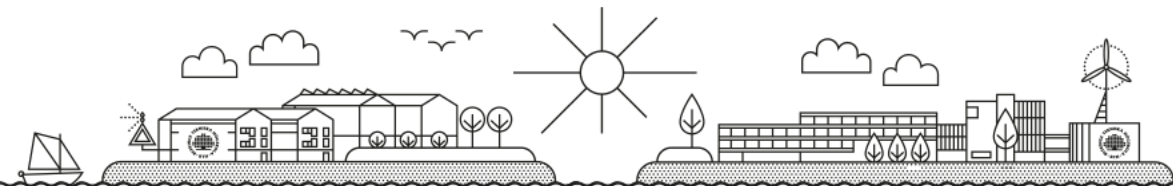
$$\sum_{i=1}^l \frac{|u|!}{i! (|u| - i)!}$$



FREQUENT ITEMSET MINING ALGORITHMS



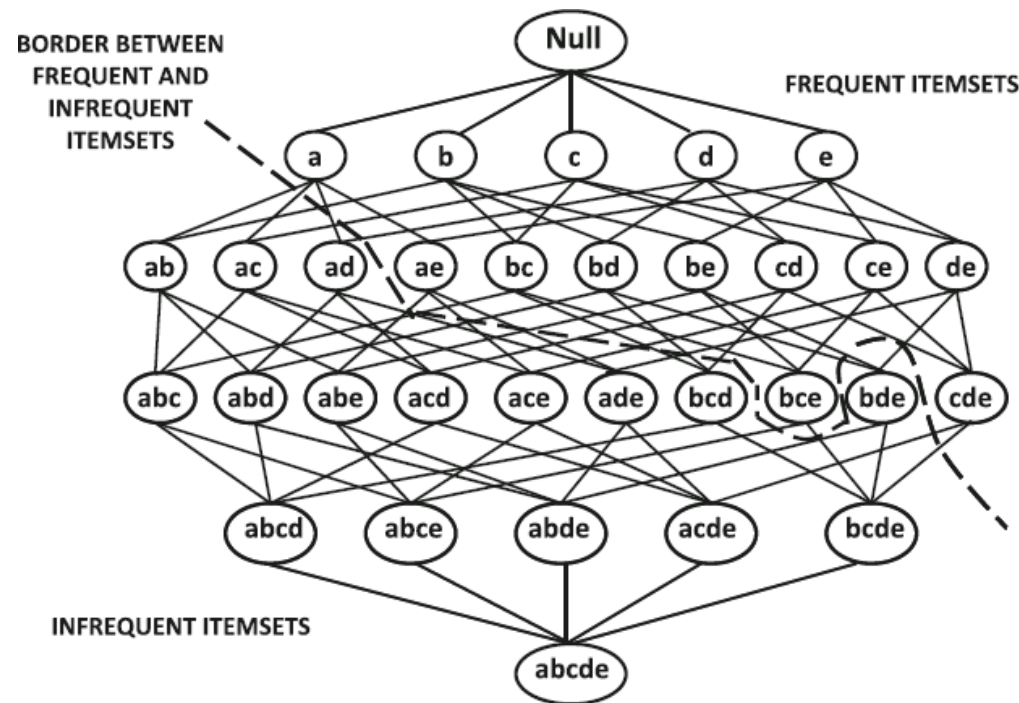
- Brute Force Algorithms
- **Apriori Algorithm**
- FP-Growth Algorithm



THE APRIORI ALGORITHM

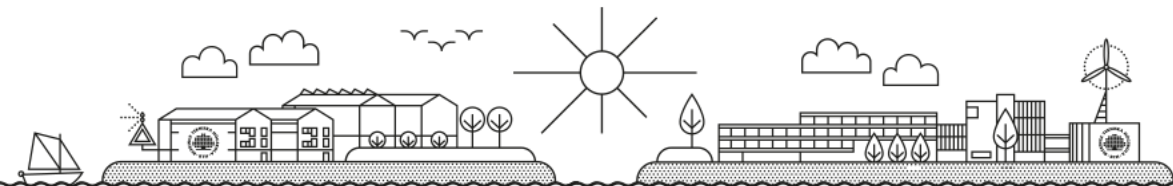
○ The Basic Idea

- Use the **downward closure property** to prune the candidate search space



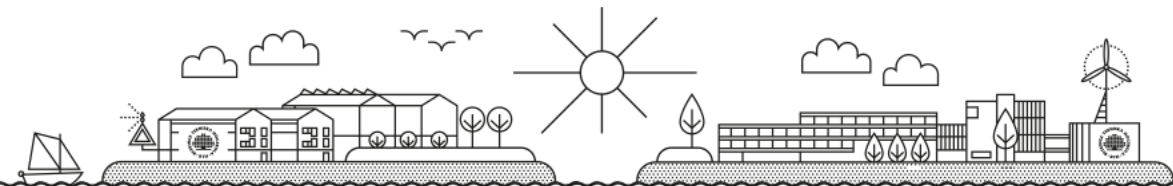
THE APRIORI ALGORITHM

- The Basic Idea
 - Use the **downward closure property** to prune the candidate search space
- The Overall Procedure (level-wise)
 1. Using the frequent k -itemsets to generate $(k + 1)$ -candidates
 2. Prune the candidates before counting
 3. Counts the supports of the remaining $(k + 1)$ -candidates
 4. Stop if there is no frequent $(k + 1)$ -itemsets



THE PSEUDOCODE

```
Algorithm Apriori(Transactions:  $\mathcal{T}$ , Minimum Support: minsup)  
begin  
   $k = 1$ ;  
   $\mathcal{F}_1 = \{ \text{All Frequent 1-itemsets} \}$ ;  
  while  $\mathcal{F}_k$  is not empty do begin  
    Generate  $\mathcal{C}_{k+1}$  by joining itemset-pairs in  $\mathcal{F}_k$ ;  
    Prune itemsets from  $\mathcal{C}_{k+1}$  that violate downward closure;  
    Determine  $\mathcal{F}_{k+1}$  by support counting on  $(\mathcal{C}_{k+1}, \mathcal{T})$  and retaining  
      itemsets from  $\mathcal{C}_{k+1}$  with support at least minsup;  
     $k = k + 1$ ;  
  end;  
  return( $\cup_{i=1}^k \mathcal{F}_i$ );  
end
```



CANDIDATES GENERATION (1)

- A Naïve Approach

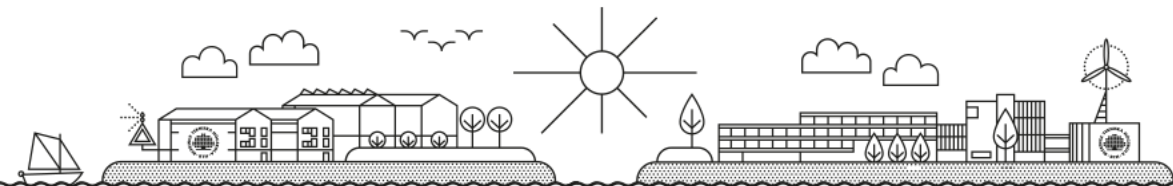
- Check all the possible combinations of frequent k -itemsets
- Keep all the $(k + 1)$ -itemsets

- An Example of the Naive Approach

- k -itemsets: $\{abc\}$ $\{bcd\}$ $\{abd\}$ $\{cde\}$
- $\{abc\} + \{bcd\} = \{abcd\}$
- $\{bcd\} + \{abd\} = \{abcd\}$
- $\{abd\} + \{cde\} = \{abcde\}$
-

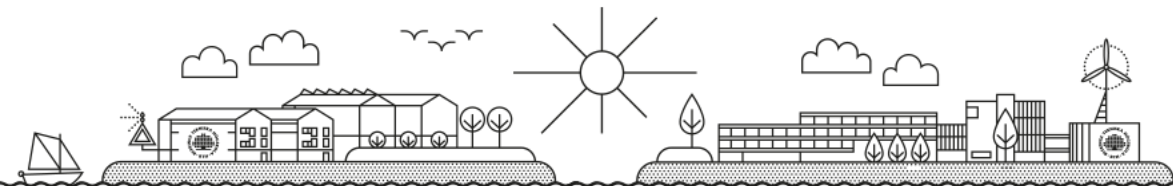
A blue starburst graphic with multiple points, containing the text "Redundancy And Inefficiency".

*Redundancy
And Inefficiency*



CANDIDATES GENERATION (2)

- Introduction of ordering
 - Items in U have a lexicographic ordering
 - Itemsets can be ordered as strings
- A Better Approach
 - Order the frequent k -itemsets
 - Merge two itemsets if the **first** $k - 1$ items of them are the same
 - E.g., $abcd$, the first two items have to be a and b
 $\underline{abc}, \underline{abd} \rightarrow \underline{abcd}$
if either abc or abd are **infrequent**, then $abcd$ **will not be generated**.



EXAMPLE (1)

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

1st scan

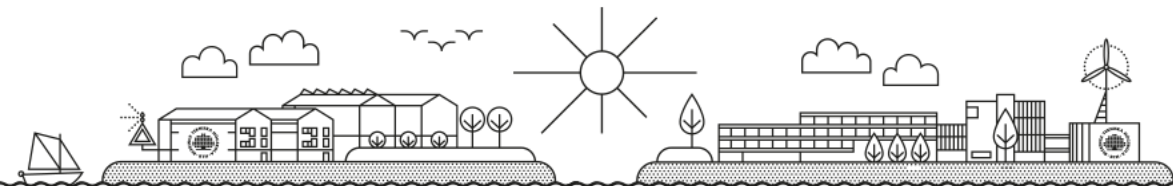
C_1

Itemset	Sup.count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

minsup = 2

F_1

Itemset	Sup.count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2



EXAMPLE (2)

F_1

Itemset	Sup.count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Generate
 C_2
candidates
from F_1
→
2nd scan

C_2

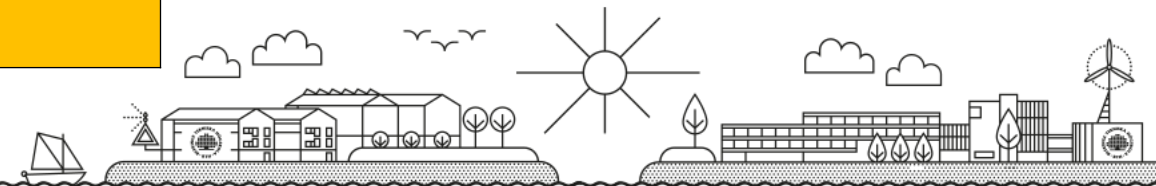
Itemset	Sup.count
{I1,I2}	4
{I1,I3}	4
{I1,I4}	1
{I1,I5}	2
{I2,I3}	4
{I2,I4}	2
{I2,I5}	2
{I3,I4}	0
{I3,I5}	1
{I4,I5}	0

minsup = 2

F_2

Itemset	Sup.count
{I1,I2}	4
{I1,I3}	4
{I1,I5}	2
{I2,I3}	4
{I2,I4}	2
{I2,I5}	2

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1,I2,I3,I5
T900	I1,I2,I3



EXAMPLE (3)

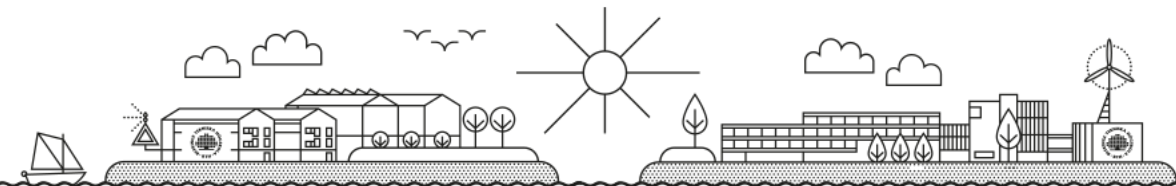
F_2

Itemset	Sup.count
{I1,I2}	4
{I1,I3}	4
{I1,I5}	2
{I2,I3}	4
{I2,I4}	2
{I2,I5}	2

- {I1,I2,I3} = {I1,I2} , {I1,I3}, {I2,I3}; will be added
- {I1,I2,I5} = {I1,I2} , {I1,I5}, {I2,I5}; will be added
- {I1,I3,I5} = {I1,I3} , {I1,I5}, {I3,I5}; won't be added
- {I2,I3,I4} = {I2,I3} , {I2,I4}, {I3,I4}; won't be added
- {I2,I3,I5} = {I2,I3} , {I2,I5}, {I3,I5}; won't be added
- {I2,I4,I5} = {I2,I4} , {I2,I5}, {I4,I5}; won't be added

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1,I2,I3,I5
T900	I1,I2,I3

- Candidate generation is done via **self-joining** and **pruning**.



EXAMPLE (4)

F_2

Itemset	Sup.count
{I1,I2}	4
{I1,I3}	4
{I1,I5}	2
{I2,I3}	4
{I2,I4}	2
{I2,I5}	2

Generate
 C_3
candidates
from F_2
3rd scan

C_3

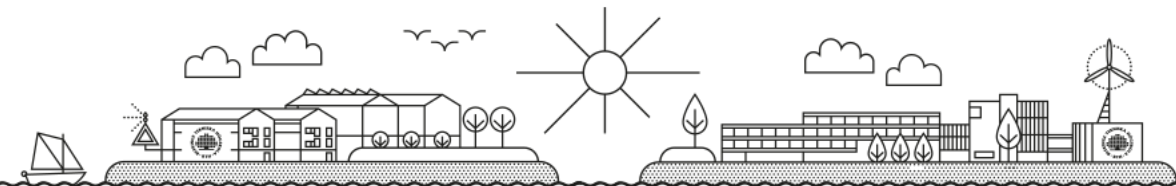
Itemset	Sup.count
{I1,I2,I3}	2
{I1,I2,I5}	2

minsup = 2

F_3

Itemset	Sup.count
{I1,I2,I3}	2
{I1,I2,I5}	2

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1,I2,I3,I5
T900	I1,I2,I3



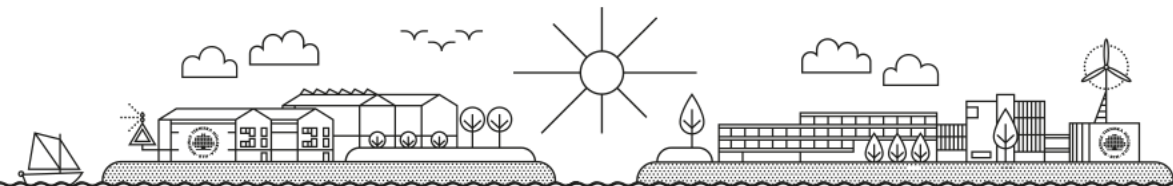
EXAMPLE (5)

F_3

Itemset	Sup.count
{I1,I2,I3}	2
{I1,I2,I5}	2

- $\{I1,I2,I3,I5\} = \{I1,I2,I3\} , \{I1,I2,I5\}, \{I2,I3,I5\}$; won't be added
- So $C_4 = \emptyset$ and the algorithm terminates

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1,I2,I3,I5
T900	I1,I2,I3



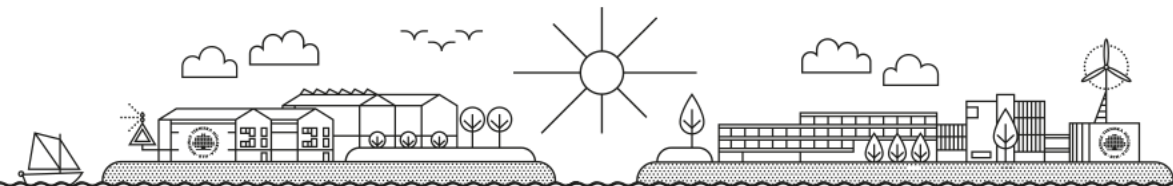
SUPPORT COUNTING

○ A Naïve Approach

- For each candidate, $I_i \in \mathcal{C}_{k+1}$
 - ✓ For each transaction T_j in the transaction database T .
 - Check whether I_i appears in T_j

○ The limitation

- Inefficient if both $|\mathcal{C}_{k+1}|$ and $|T|$ are very large



SUPPORT COUNTING (2)

○ A Better Approach

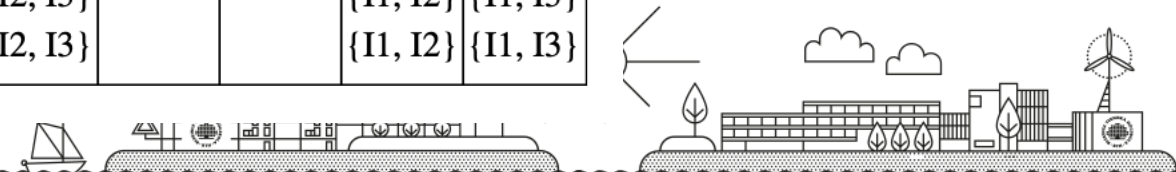
➤ Hash-based technique

- Can be used to reduce the size of C_k for $k > 1$
- $\{I1, I4\} = (1 \times 10 + 4) \bmod 7 = 0 \rightarrow$ bucket 0, bucket count 1
-
- $\{I3, I5\} = 30 + 5 \bmod 7 = 0 \rightarrow$ bucket 0, bucket count $1+1 = 2$

Create hash table H_2
using hash function
 $h(x, y) = ((\text{order of } x) \times 10 + (\text{order of } y)) \bmod 7$

H_2							
bucket address	0	1	2	3	4	5	6
bucket count	2	2	4	2	2	4	4
bucket contents	{I1, I4} {I3, I5}	{I1, I5}	{I2, I3} {I2, I3} {I2, I3}	{I2, I4} {I2, I4}	{I2, I5} {I2, I5}	{I1, I2} {I1, I2} {I1, I2}	{I1, I3} {I1, I3} {I1, I3}

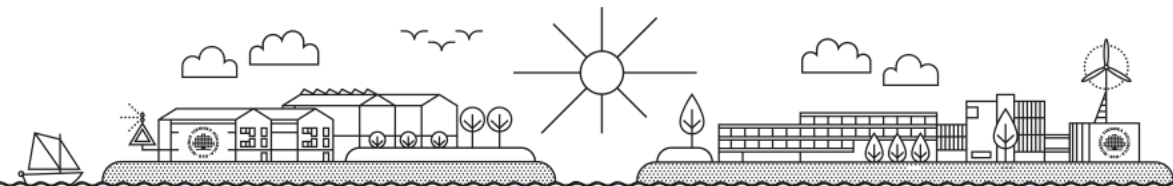
For **minsup = 3**,
buckets 0, 1, 3, and 4
can be removed



FREQUENT ITEMSET MINING ALGORITHMS

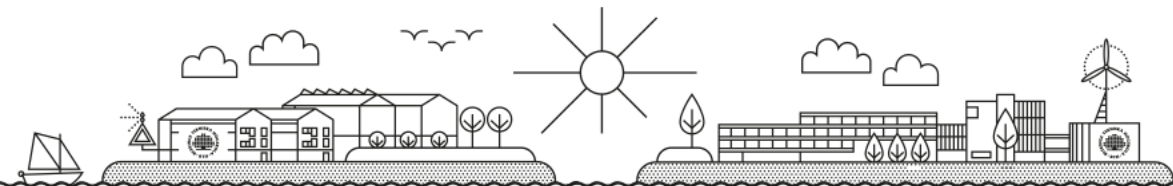


- Brute Force Algorithms
- Apriori Algorithm
- **FP-Growth Algorithm**



PATTERN-GROWTH APPROACH: MINING FREQUENT PATTERNS WITHOUT CANDIDATE GENERATION

- Bottlenecks of the Apriori approach
 - Breadth-first (i.e., level-wise) search
 - Candidate generation and test
 - Often generates a huge number of candidates
- The Frequent Pattern (FP)-Growth Approach
 - Depth-first search
 - Avoid explicit candidate generation
- Major philosophy: Grow long patterns from short ones using local frequent items only
 - $\{abc\}$ is a frequent pattern
 - Get all transactions having $\{abc\}$, i.e., project DB on $\{abc\}$: $DB|abc$
 - $\{d\}$ is a local frequent item in $DB|abc \rightarrow \{abcd\}$ is a frequent pattern



THE PSEUDOCODE



Algorithm *FP-growth*(FP-Tree of frequent items: $\mathcal{FPT}$, Minimum Support: $minsup$,
Current Suffix: P)

begin

if $\mathcal{FPT}$ is a single path

then determine all combinations C of nodes on the
 path, and report $C \cup P$ as frequent;

else (Case when $\mathcal{FPT}$ is not a single path)

for each item i in $\mathcal{FPT}$ **do begin**

report itemset $P_i = \{i\} \cup P$ as frequent;

 Use pointers to extract conditional prefix paths
 from $\mathcal{FPT}$ containing item i ;

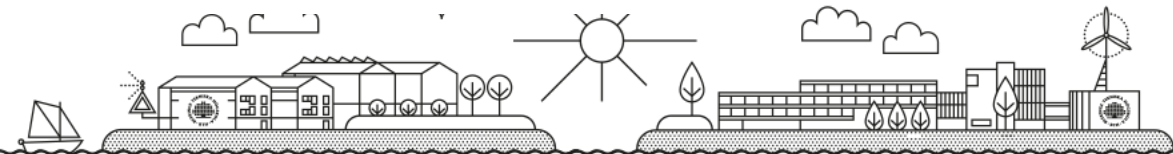
 Readjust counts of prefix paths and remove i ;

 Remove infrequent items from prefix paths and reconstruct
 conditional FP-Tree $\mathcal{FPT}_i$;

if ($\mathcal{FPT}_i \neq \phi$) **then** *FP-growth*($\mathcal{FPT}_i$, $minsup$, P_i);

end

end



CONSTRUCT FP-TREE FROM A TRANSACTION DATABASE

<u>TID</u>	<u>Items bought</u>	<u>Ordered, frequent items</u>
1	{f, a, c, d, g, i, m, p}	
2	{a, b, c, f, l, m, o}	
3	{b, f, h, j, o, w}	
4	{b, c, k, s, p}	
5	{a, f, c, e, l, p, m, n}	

1. Scan DB once, find single item frequent patterns:

minsup = 3

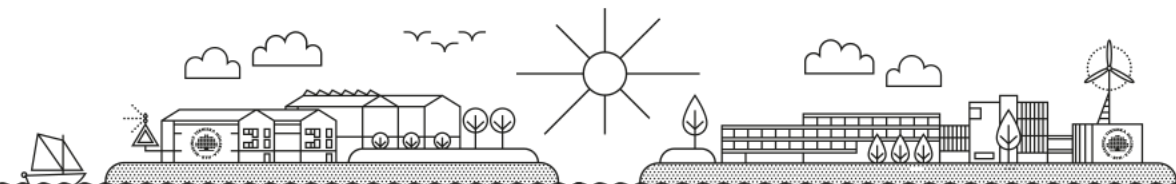
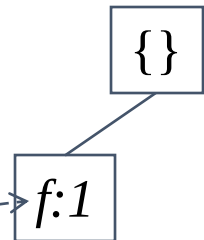
f:4, a:3, c:4, b:3, m:3, p:3

2. Sort frequent items in frequency descending order,
f-list

f-list = f-c-a-b-m-p

3. Scan DB again, construct FP-tree

Header Table	
<u>Item</u>	<u>frequency head</u>
f	4
c	4
a	3
b	3
m	3
p	3



CONSTRUCT FP-TREE FROM A TRANSACTION DATABASE

<i>TID</i>	<i>Items bought</i>	<i>Ordered, frequent items</i>
1	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
2	{a, b, c, f, l, m, o}	{f, c, a, b, m}
3	{b, f, h, j, o, w}	{f, b}
4	{b, c, k, s, p}	{c, b, p}
5	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}

1. Scan DB once, find single item frequent patterns:

minsup = 3

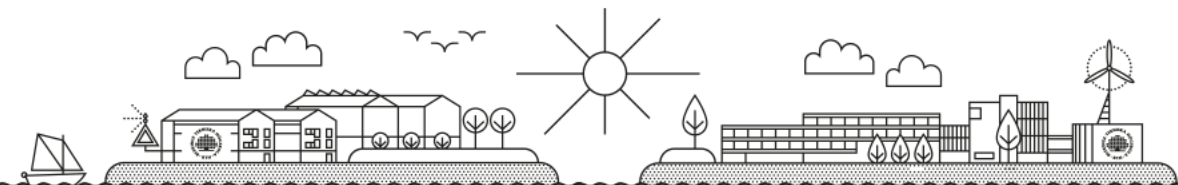
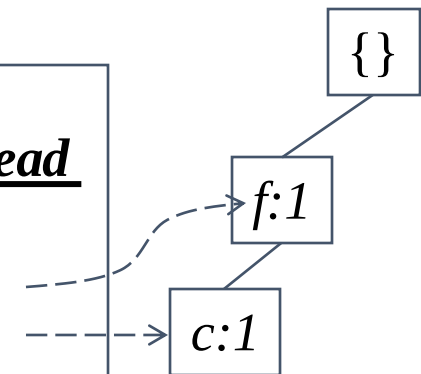
f:4, a:3, c:4, b:3, m:3, p:3

2. Sort frequent items in frequency descending order, f-list

f-list = f-c-a-b-m-p

3. Scan DB again, construct FP-tree

Header Table	
<u>Item frequency head</u>	
f	4
c	4
a	3
b	3
m	3
p	3



CONSTRUCT FP-TREE FROM A TRANSACTION DATABASE

<i>TID</i>	<i>Items bought</i>	<i>Ordered, frequent items</i>
1	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
2	{a, b, c, f, l, m, o}	{f, c, a, b, m}
3	{b, f, h, j, o, w}	{f, b}
4	{b, c, k, s, p}	{c, b, p}
5	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}

1. Scan DB once, find single item frequent patterns:

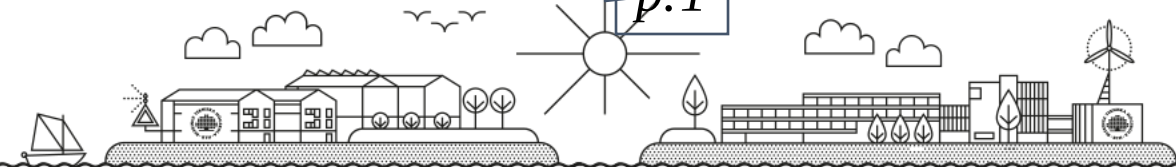
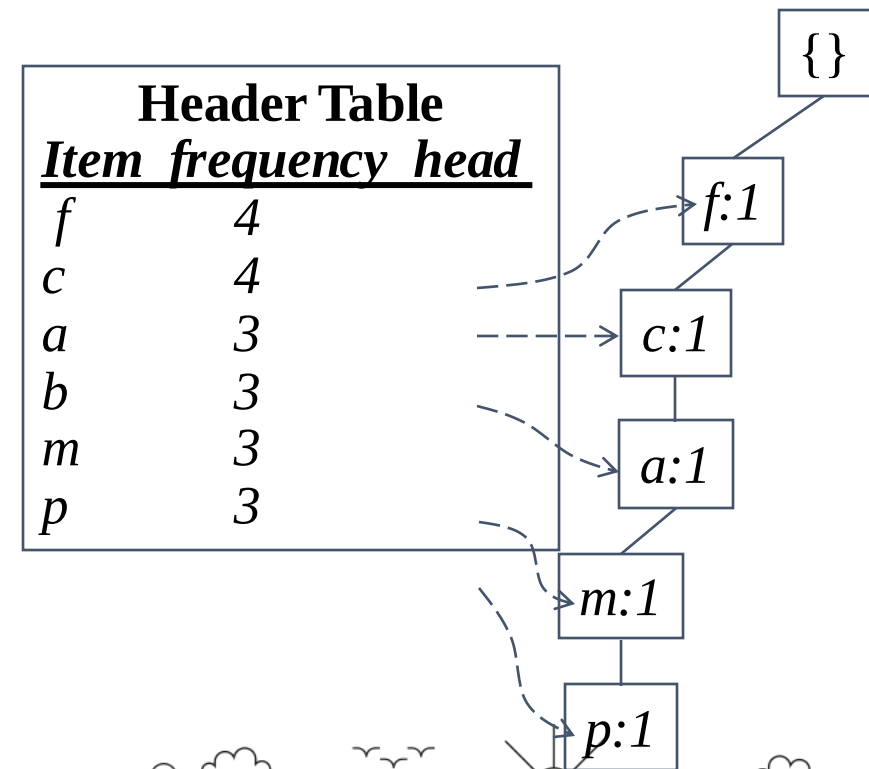
minsup = 3

f:4, a:3, c:4, b:3, m:3, p:3

2. Sort frequent items in frequency descending order,
f-list

f-list = f-c-a-b-m-p

3. Scan DB again, construct FP-tree



CONSTRUCT FP-TREE FROM A TRANSACTION DATABASE

<i>TID</i>	<i>Items bought</i>	<i>Ordered, frequent items</i>
1	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
2	{a, b, c, f, l, m, o}	{f, c, a, b, m}
3	{b, f, h, j, o, w}	{f, b}
4	{b, c, k, s, p}	{c, b, p}
5	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}

1. Scan DB once, find single item frequent patterns:

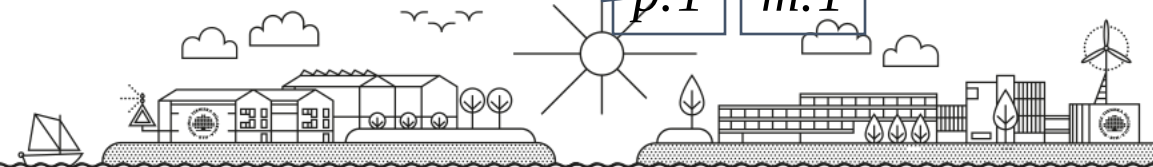
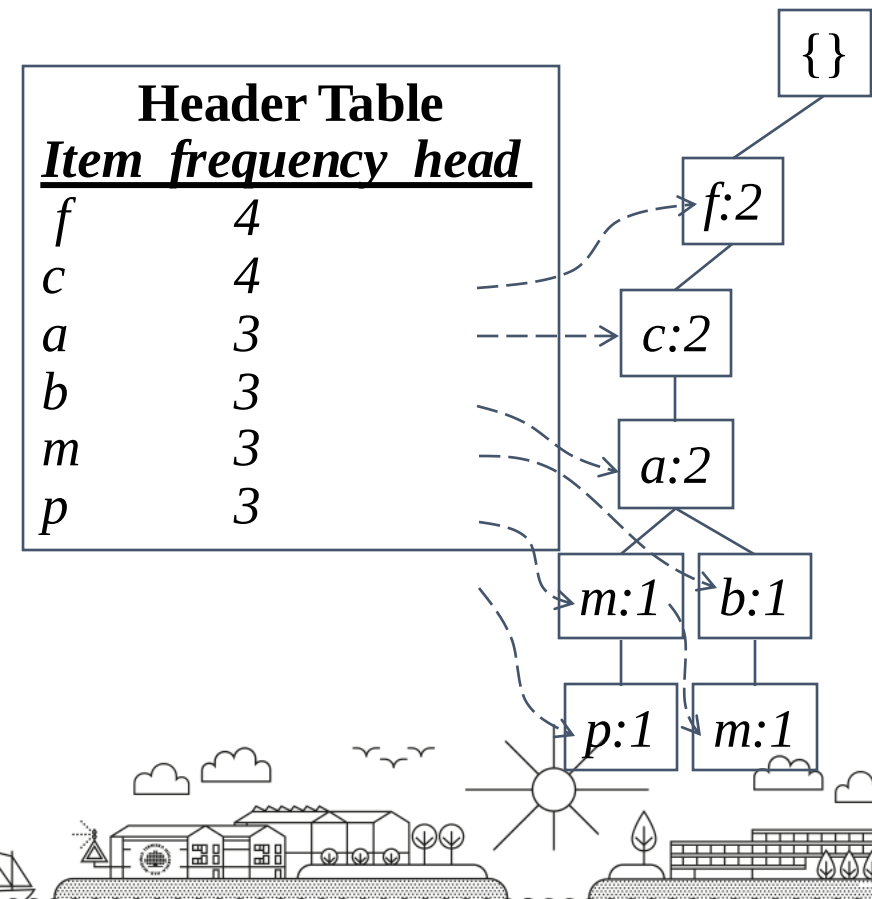
minsup = 3

f:4, a:3, c:4, b:3, m:3, p:3

2. Sort frequent items in frequency descending order, f-list

f-list = f-c-a-b-m-p

3. Scan DB again, construct FP-tree



CONSTRUCT FP-TREE FROM A TRANSACTION DATABASE

<i>TID</i>	<i>Items bought</i>	<i>Ordered, frequent items</i>
1	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
2	{a, b, c, f, l, m, o}	{f, c, a, b, m}
3	{b, f, h, j, o, w}	{f, b}
4	{b, c, k, s, p}	{c, b, p}
5	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}

1. Scan DB once, find single item frequent patterns:

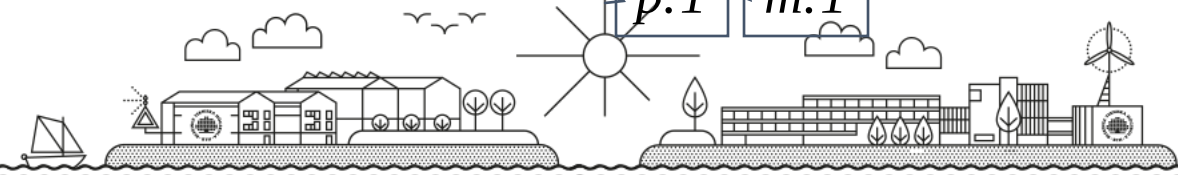
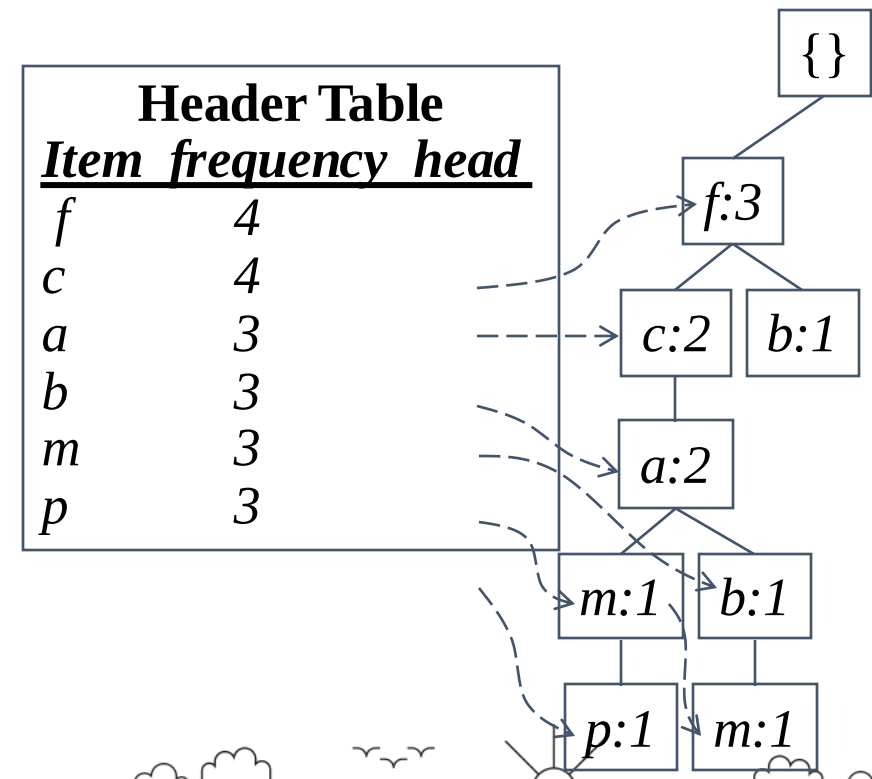
minsup = 3

f:4, a:3, c:4, b:3, m:3, p:3

2. Sort frequent items in frequency descending order, f-list

f-list = f-c-a-b-m-p

3. Scan DB again, construct FP-tree



CONSTRUCT FP-TREE FROM A TRANSACTION DATABASE

<i>TID</i>	<i>Items bought</i>	<i>Ordered, frequent items</i>
1	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
2	{a, b, c, f, l, m, o}	{f, c, a, b, m}
3	{b, f, h, j, o, w}	{f, b}
4	{b, c, k, s, p}	{c, b, p}
5	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}

1. Scan DB once, find single item frequent patterns:

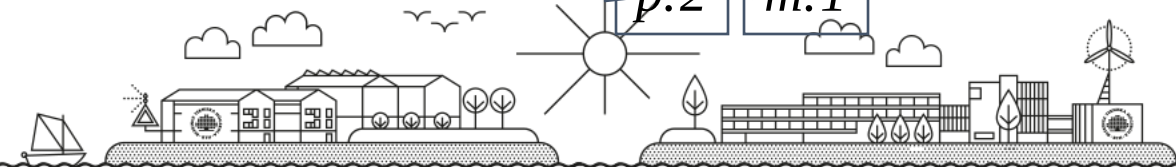
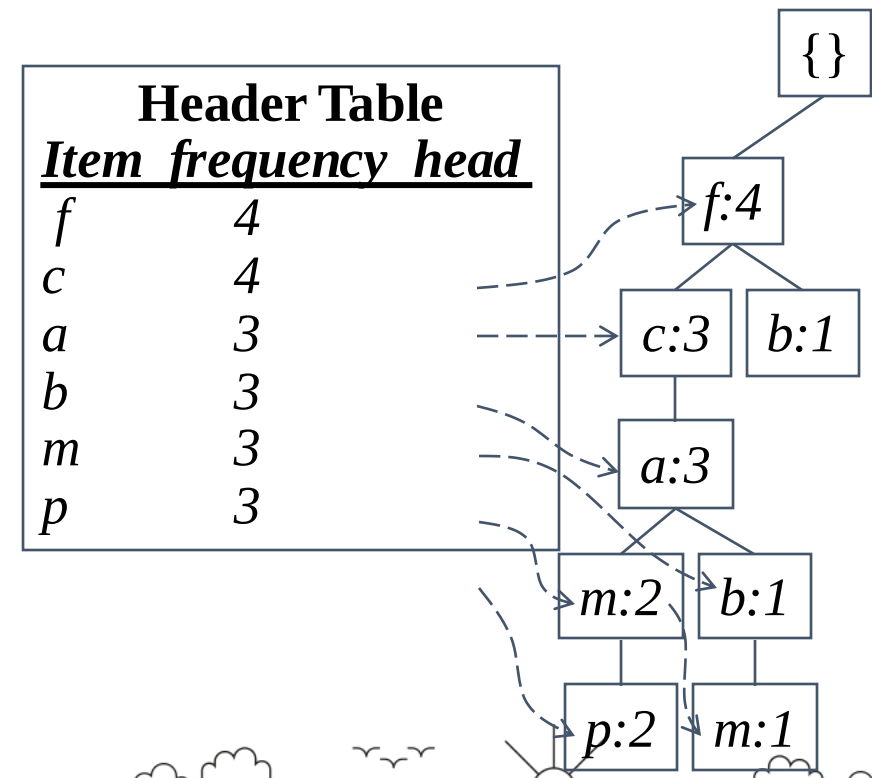
minsup = 3

f:4, a:3, c:4, b:3, m:3, p:3

2. Sort frequent items in frequency descending order,
f-list

f-list = f-c-a-b-m-p

3. Scan DB again, construct FP-tree



CONSTRUCT FP-TREE FROM A TRANSACTION DATABASE

<i>TID</i>	<i>Items bought</i>	<i>Ordered, frequent items</i>
1	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
2	{a, b, c, f, l, m, o}	{f, c, a, b, m}
3	{b, f, h, j, o, w}	{f, b}
4	{b, c, k, s, p}	{c, b, p}
5	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}

1. Scan DB once, find single item frequent patterns:

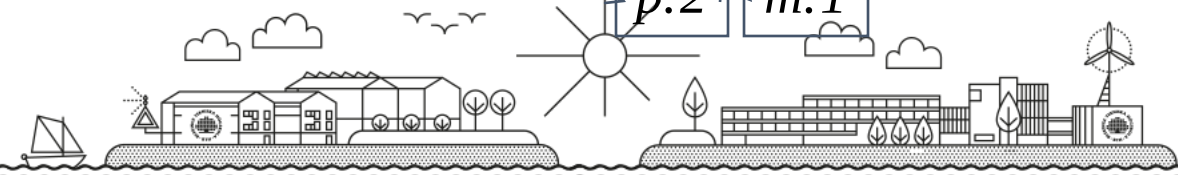
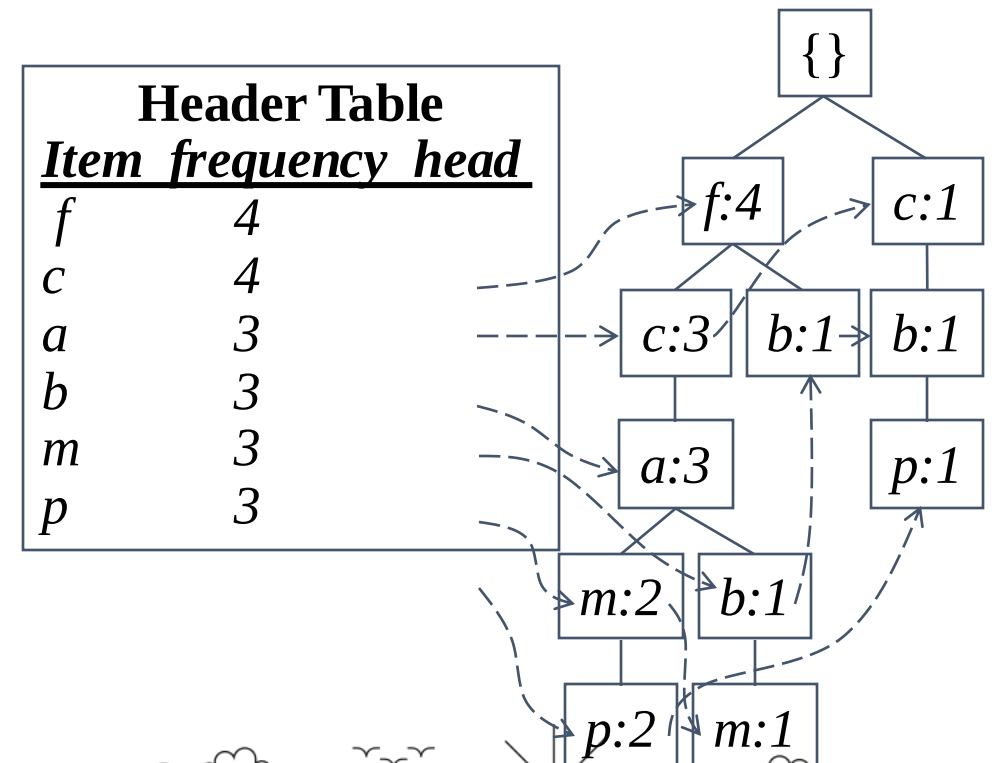
minsup = 3

f:4, a:3, c:4, b:3, m:3, p:3

2. Sort frequent items in frequency descending order, f-list

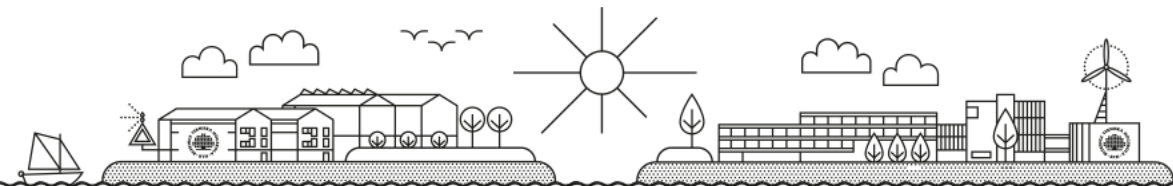
f-list = f-c-a-b-m-p

3. Scan DB again, construct FP-tree



PARTITION PATTERNS AND DATABASES

- Frequent patterns can be partitioned into subsets according to f-list
 - f-list = f-c-a-b-m-p
 - Patterns containing p
 - Patterns having m but no p
 - ...
 - Patterns having c but no a nor b, m, p
 - Pattern f
- Completeness and non-redundancy

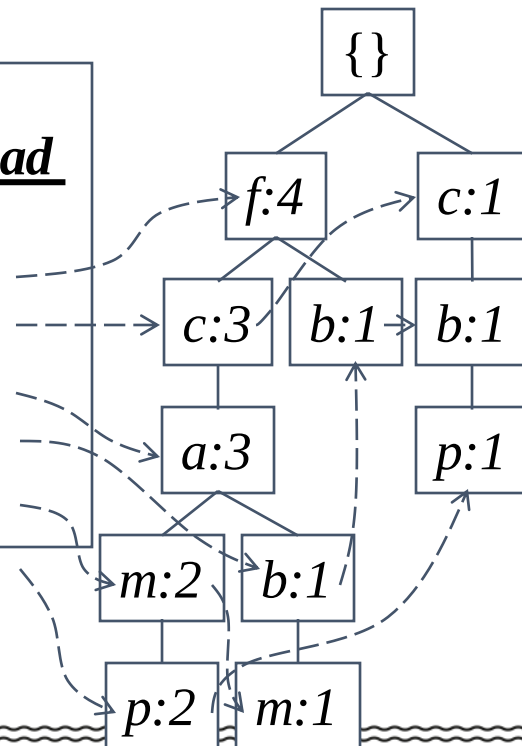


FIND PATTERNS HAVING P FROM P-CONDITIONAL DATABASE

- Starting at the frequent item header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item p
- Accumulate all of *transformed prefix paths* of item p to form p 's conditional pattern base

minsup = 3

Header Table		
<i>Item</i>	<i>frequency</i>	<i>head</i>
f	4	
c	4	
a	3	
b	3	
m	3	
p	3	



Conditional pattern bases

item *cond. pattern base*

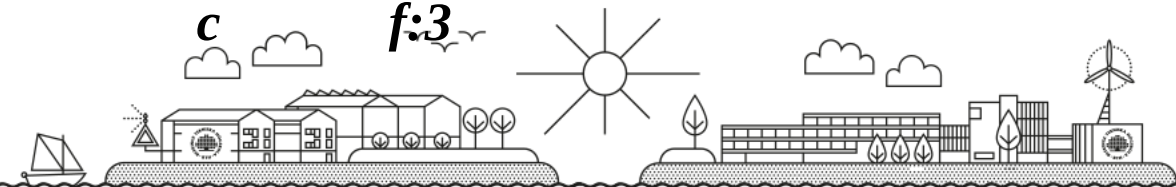
p $fcam:2, cb:1$

m $fca:2, fcab:1$

b $fca:1, f:1, c:1$

a $fc:3$

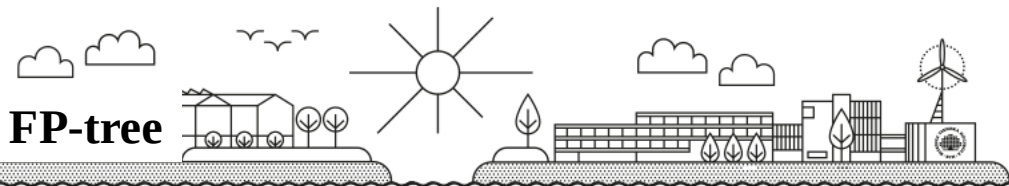
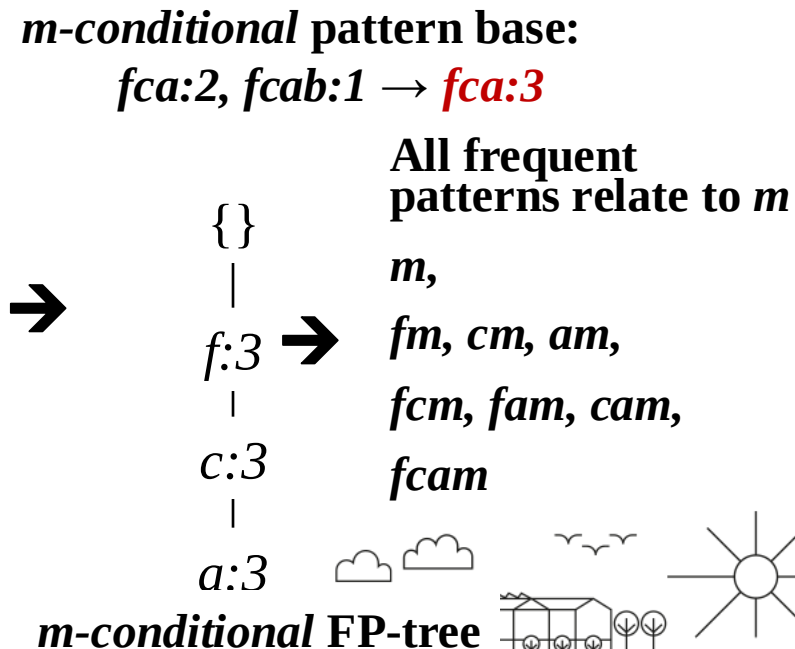
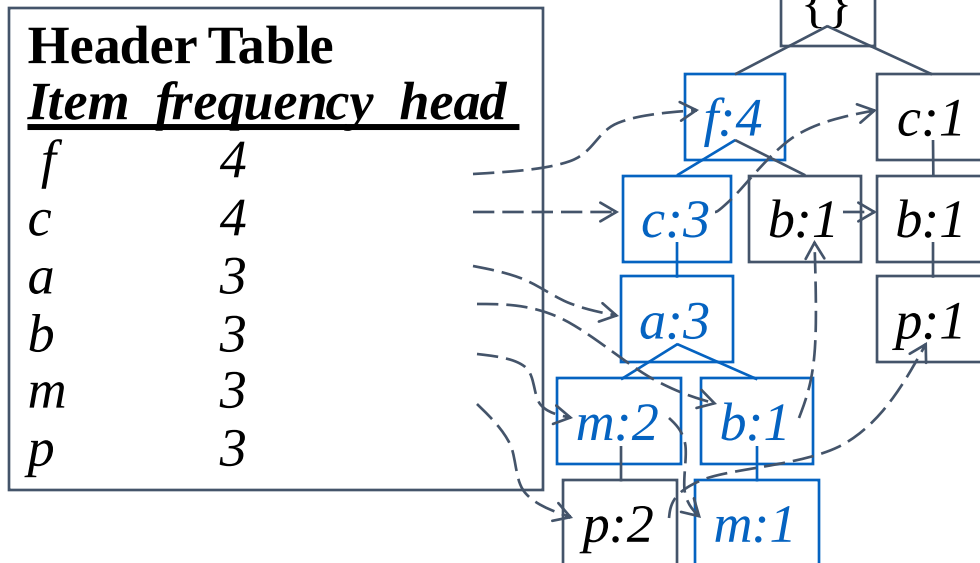
c $f:3$



FROM CONDITIONAL PATTERN-BASES TO CONDITIONAL FP-TREES

- For each pattern-base
 - Accumulate the count for each item in the base
 - Construct the FP-tree for the frequent items of the pattern base

minsup = 3

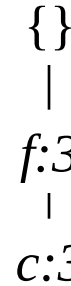


RECURSION: MINING EACH CONDITIONAL FP-TREE



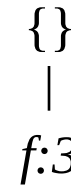
m-conditional FP-tree

Cond. pattern base of "am": (fc:3)



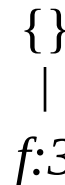
am-conditional FP-tree

Cond. pattern base of "cm": (f:3)

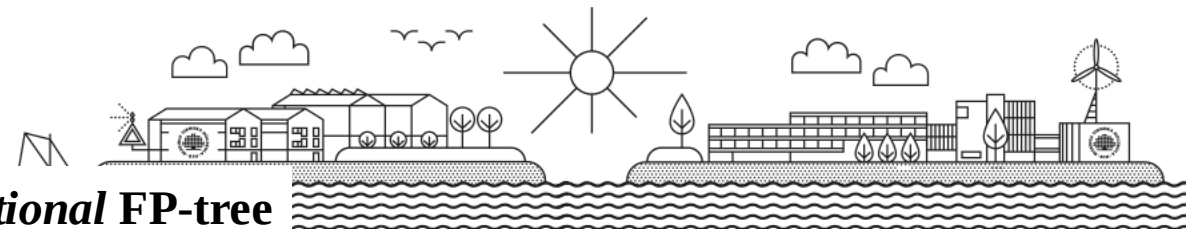


cm-conditional FP-tree

Cond. pattern base of "cam": (f:3)

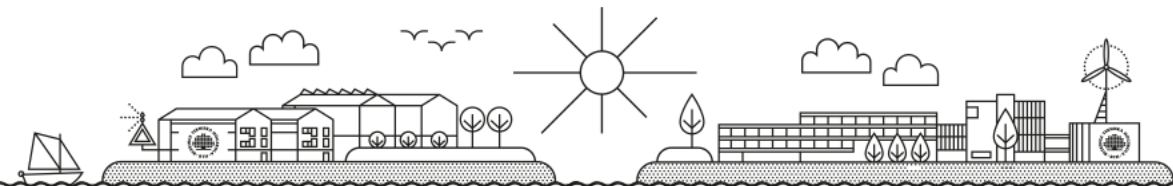


cam-conditional FP-tree



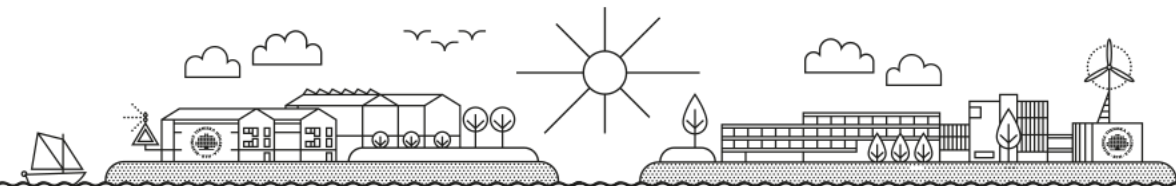
USEFUL META-ALGORITHMS

- **Meta-algorithm:** An algorithm that uses a particular algorithm as a subroutine
 - either to make the original algorithm more efficient (e.g., by sampling)
 - or to gain new insights
- Sampling Methods
- Data Partitioned Ensembles
- Generalization to Other Data Types



SAMPLING METHODS

- The Procedure
 - Sample a subset of the transactions
 - Apply mining algorithm to sampled data
- Challenges
 - **False positives:** These are patterns that meet the support threshold on the sample but not on the whole dataset.
 - Post-processing: scanning the data once
 - **False negatives:** These are patterns that do not meet the support threshold on the sample but meet the threshold on the whole dataset.
 - Reduce the support threshold.



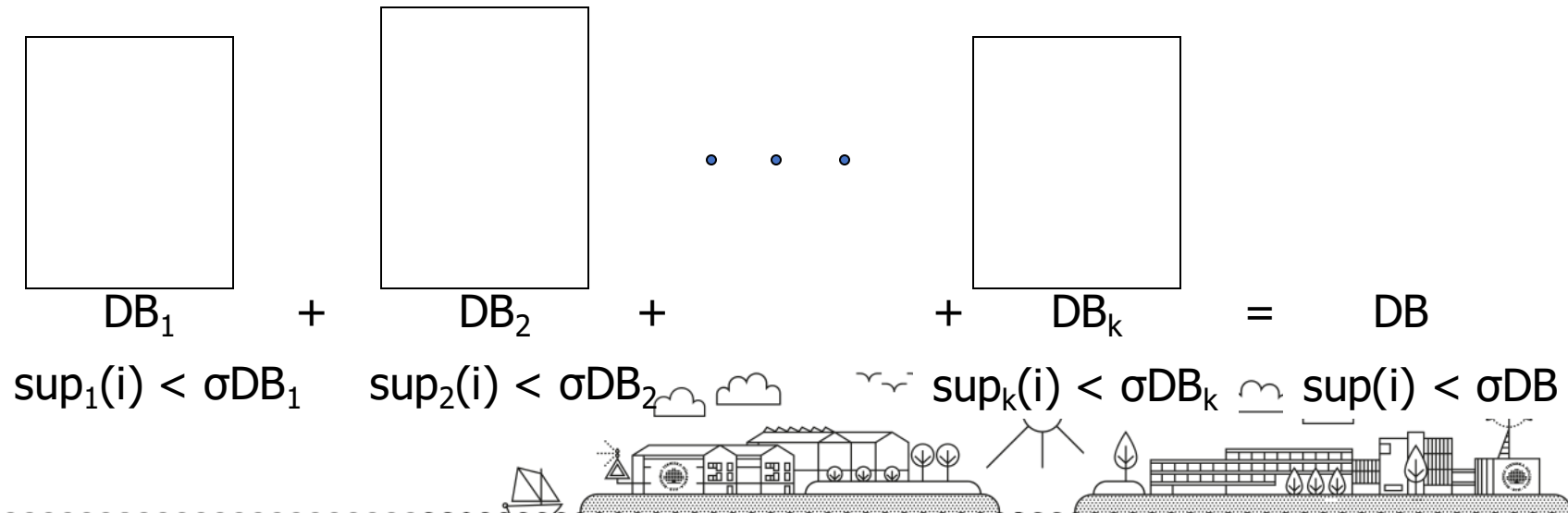
DATA PARTITIONED ENSEMBLES

○ The Procedure

- The transaction database is partitioned into k disjoint segments
- The mining algorithm is independently applied to each of these k segments
- Post-processing of support counting to remove false positives
 - every frequent pattern must appear in at least one of the segments

○ Property

- No false negatives



GENERALIZATION TO OTHER DATA TYPES

- Quantitative Data

- Rules contain quantitative attributes

$(Age = 90) \Rightarrow Checkers.$

- This rule may not have enough support for **each individual**

$Age[85, 95] \Rightarrow Checkers.$

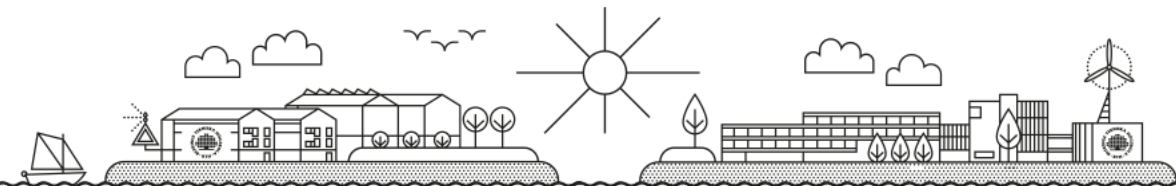
- Discretize and convert to binary form

- Categorical Data

- Rules contain mixed attributes

$(Gender = Male), Age[20, 30] \Rightarrow Basketball.$

- Transform to binary values



REFERENCES

- [1]. Agrawal, Rakesh, Tomasz Imieliński, and Arun Swami. "Mining association rules between sets of items in large databases." *Proceedings of the 1993 ACM SIGMOD international conference on Management of data*. 1993. (Apriori)
- [2]. Han, Jiawei, et al. "Mining frequent patterns without candidate generation: A frequent-pattern tree approach." *Data mining and knowledge discovery* 8.1 (2004): 53-87. (PF-Growth)
- [3]. Han, Jiawei, Jian Pei, and Hanghang Tong. *Data mining: concepts and techniques*. Morgan kaufmann, 2022.
- [4]. Aggarwal, Charu C. *Data mining: the textbook*. Vol. 1. New York: springer, 2015.

